CloudSentinel AI: A Context-Aware Framework for Cloud Misconfiguration Risk Assessment and Candidate Attack-Path Analysis

Harshith

***Abstract*—Enterprise cloud infrastructures routinely suffer from finding fatigue, where thousands of isolated resource alerts generated by standard auditing suites obscure actual threat exposure. This paper introduces CloudSentinel AI, an AWS-focused architecture engineered to shift cloud posture analysis from disjointed resource checklists to relationship-aware graph reasoning. Our design ingests read-only configuration metadata across compute, storage, identity, and network boundaries, mapping assets and dependencies into a typed security knowledge graph. Rather than assuming theoretical connectivity equates to real-world compromise, our candidate attack-path engine enforces strict precondition validation before asserting multi-hop traversal sequences. We formulate an interpretable, four-factor scoring heuristic that balances finding severity against topological path context, external network reachability, and asset business value. To streamline analyst triage without introducing generative hallucinations, our reporting workflow binds Large Language Models behind deterministic configuration facts and mandatory human review. We validate the framework's core assumptions against a verified, non-production AWS baseline deployed in the `ap-south-2` region. Preliminary read-only scans successfully flagged an open security-group rule and a scoped IAM wildcard while correctly refusing to assert candidate attack paths due to missing network interfaces, absent Internet routes, and active account-level S3 protections. We establish an empirical evaluation protocol across six representative cloud scenarios, demonstrating bounded evidence capture and defensive precondition rejection.**

***Index Terms*—cloud security, cloud misconfiguration, AWS, knowledge graph, attack-path analysis, risk prioritisation, LLM-assisted security reporting**

I. Introduction

Modern software engineering has largely abandoned on-premises perimeters in favor of software-defined, multi-tenant cloud ecosystems [1]. While on-demand resource provisioning delivers immense elasticity, it also relocates the operational attack surface to an interconnected fabric of identity policies, network boundaries, and storage permissions [2], [3]. In these environments, security posture is not an intrinsic property of an isolated asset. Instead, risk emerges from transitive relationships among loosely coupled cloud services [4], [5].

Industry incident retrospectives show that cloud breaches rarely originate from zero-day binary exploitation. Rather, real-world failures are driven by mundane administrative oversights, overly permissive Identity and Access Management (IAM) permissions, and configuration flaws embedded in Infrastructure-as-Code (IaC) deployment definitions [6], [7]. Contemporary Cloud Security Posture Management (CSPM) suites evaluate cloud tenancies by matching individual resources against static compliance benchmarks. This paradigm generates vast volumes of atomized alerts, leaving security operations centers (SOC) overwhelmed by warning fatigue [8]. An isolated finding carries little diagnostic value without its operational environment: an open ingress rule on an Amazon EC2 security group represents negligible practical risk if the instance resides in an unrouted private subnet, whereas an identical rule attached to an Internet-facing host with broad assume-role capabilities constitutes an immediate escalation path.

To bridge this contextual void, we present CloudSentinel AI, an AWS-targeted framework that unifies multi-service metadata extraction with relationship-aware graph analytics. Rooted in the Design Science Research (DSR) paradigm [9], our work treats the proposed framework as an engineering artifact evaluated against clear operational bounds. Specifically, this paper contributes:

1) A multi-service AWS ontology and graph schema that models compute, storage, identity, and network resources as typed nodes and inter-service dependencies as directed edges.
2) A candidate attack-path discovery engine that enumerates bounded traversal chains while discarding paths lacking verified runtime preconditions.
3) A linear, fully explainable risk scoring heuristic that quantifies exposure from raw severity, asset criticality, candidate-path context, and network accessibility.
4) An evidence-bound LLM reporting contract that generates human-reviewed triage summaries strictly constrained to collected configuration evidence.

Importantly, we present CloudSentinel AI as a formal design and validation protocol. We deliberately avoid asserting premature production superiority or general detection claims ahead of controlled benchmark execution.

II. Background and Related Work

A. Cloud Misconfiguration Detection and IaC Verification

Configuration defects are widely acknowledged as the predominant operational hazard across public cloud tenancies [6]. A rich body of research focuses on shifting security left by analyzing declarative deployment templates prior to provisioning. Rahman et al. [7] cataloged seven recurring security smells in IaC scripts, showing that hardcoded credentials and excessive privileges regularly bypass code review. Chiari et al. [10] surveyed static analysis methodologies for

IaC, contrasting pre-deployment syntax validation against the realities of live cloud drift. In container ecosystems, empirical studies by Bose et al. [11] and Rahman et al. [12] identified recurring RBAC misconfigurations and unsegmented pod networks within open-source Kubernetes manifests.

At the storage layer, public object exposures represent an ongoing vulnerability. Olorunlana [13] introduced an automated policy-enforcement framework to block insecure Amazon S3 bucket configurations directly within deployment templates. However, static linters cannot observe post-deployment drift, dynamic trust relationships, or cross-service interactions in active enterprise accounts. Addressing this gap, Torkura et al. [4] proposed continuous auditing architectures for multi-cloud setups, emphasizing real-time telemetry analysis. In parallel, van Ede et al. [8] utilized anomaly detection to spot risky permission combinations in AWS IAM policies. Yet, analyzing identity definitions in isolation from network topologies fails to reveal whether an overly permissive principal is practically reachable by an attacker.

B. Graph-Based Modeling and Attack-Path Analysis

Knowledge graphs provide a robust mathematical abstraction for representing heterogeneous entities and complex inter-resource relationships [14]. In network security, graph structures have long underpinned vulnerability analysis. Phillips and Swiler [15] pioneered graph-based vulnerability assessment, laying the groundwork for algorithmic attack-graph generation. Sheyner et al. [16] formalized attack-graph derivation using model checking, while Jha et al. [17] applied formal cut-set logic to identify optimal remediation points.

Adapting these concepts to cloud fabrics requires addressing dynamic virtualization, transient workloads, and identity-based perimeters. Sabur et al. [18] tackled combinatorial state-space growth in cloud networks through modular graph decomposition. Elmiger et al. [19] demonstrated the power of graph analytics in enterprise Microsoft tenants, identifying privilege-escalation routes that bypassed standard audit rules. In audit telemetry, Kurniawan et al. [20] introduced KRYSTAL, combining knowledge graphs with tactical attack pattern mapping. While these contributions confirm the value of graph models, they often presume that topological connectivity directly implies exploitability. CloudSentinel AI enforces a strict distinction: every graph transition is treated merely as a hypothesis until verified against explicit reachability and identity preconditions.

C. Machine Learning, Explainable AI, and Large Language Models

Machine learning methods have been extensively investigated for cloud anomaly detection and intrusion classification, as synthesized by Nassif et al. [21]. Nonetheless, deep black-box architectures offer little interpretability, preventing security teams from understanding the root cause of an alert. Explainable Artificial Intelligence (XAI) has thus become essential in security operations [22]. Charmet et al. [23] highlighted that without transparent attribution, analysts struggle to justify high-impact remediation decisions. Model-agnostic explainability frameworks like LIME [24] and SHAP [25] underscore the necessity of transparent feature attribution, a principle we mirror by adopting an additive, inspectable scoring model.

Simultaneously, Large Language Models (LLMs) built upon Transformer architectures [26] have revealed remarkable zero- and few-shot reasoning capabilities [27], with systems such as GPT-4 [28] demonstrating proficiency in distilling complex technical records. In a systematic review, Xu et al. [29] surveyed LLM utility across cyber defense, identifying strong potential in log synthesis and remediation guidance. However, unconstrained LLMs exhibit dangerous tendencies toward factual hallucination and invalid security advice [29]. CloudSentinel AI mitigates this liability by confining the language model to an advisory role, strictly bounded by structured evidence and human sign-off.

D. Problem Formulation and Research Gaps

Our literature analysis identifies three unresolved deficiencies:

1) Contemporary CSPMs evaluate resources as isolated silos, failing to track multi-stage traversal risks.
2) Theoretical cloud attack-graph models often omit operational preconditions, generating unrealistic lateral movement paths.
3) Emerging LLM security tools lack deterministic ground-truth gating, introducing generative hallucinations into remediation workflows.

CloudSentinel AI directly tackles these limitations through a hybrid architecture combining deterministic rule verification, precondition-gated graph traversals, inspectable risk scoring, and evidence-locked LLM reporting.

III. Proposed Framework

A. Scope and Threat Model

CloudSentinel AI targets core infrastructure services within an AWS tenancy: Amazon EC2, Amazon S3, AWS IAM, Amazon VPC (including security groups and route tables), Amazon RDS, AWS Secrets Manager, and AWS CloudTrail. The system operates strictly via read-only AWS API credentials, collecting only accessible configuration metadata.

We define an attacker model initiated at an explicit entry condition, such as an unauthenticated external network boundary or an assumed testing principal. Attack transitions are permitted solely when substantiated by collected configuration metadata. The engine never presumes exploit availability, zero-day vulnerabilities, or unverified credential theft; every transition edge demands recorded and verifiable preconditions.

B. Architecture and Graph Schema

The processing pipeline executes through five modular phases:

- **Metadata Collection:** Gathers configuration snapshots from target AWS services via paginated read-only APIs.

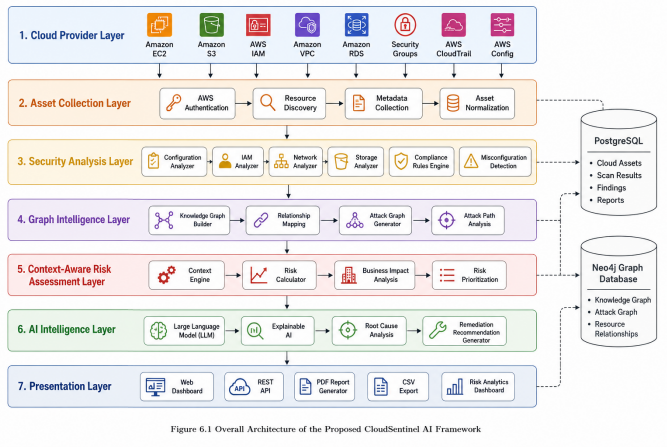


Figure 6.1 Overall Architecture of the Proposed CloudSentinel AI Framework

Fig. 1. High-level CloudSentinel AI architecture. The figure illustrates the proposed pipeline; it is not an implementation-performance result.

TABLE I
MINIMUM PROPOSED GRAPH SCHEMA

Element	Required attributes or semantics
Nodes	AWS resource, identity, policy, security finding, network boundary, or data asset; each has an account, region, source timestamp, and evidence identifier [14].
Edges	Typed relationship such as *attached-to*, *permits*, *can-reach*, *can-assume*, *stores*, or *has-finding*; every edge identifies its source metadata [20].
Findings	Rule identifier, affected node, observed evidence, severity mapping, and collection time.
Path record	Ordered edges, entry condition, required preconditions, target, supporting findings, and review status.

- **Rule Evaluation:** Evaluates deterministic security rules adapted from established benchmarks (e.g., CIS AWS Foundations).
- **Knowledge Graph Synthesis:** Translates raw metadata and findings into a typed property graph representing assets, policies, and network attachments.
- **Candidate-Path Engine:** Traverses the graph from documented entry nodes to sensitive resources, filtering out paths with unfulfilled preconditions.
- **Advisory Reporting:** Synthesizes path evidence and prioritization metrics into structured, human-reviewed incident briefs.

Fig. 1 presents the high-level system flow, and Table I specifies the minimum graph schema.

C. Candidate-Path Analysis and Prioritisation

A graph edge represents an observable relationship or permission grant; it does not constitute proof of active compromise [16], [18]. We define a candidate path as an acyclic, evidence-backed chain connecting an entry point to a high-value asset [19]. To avoid combinatorial explosion in large cloud environments, the search engine enforces a maximum traversal depth d and retains at most k candidate paths per target. Paths missing requisite preconditions (such as unroutable subnets or absent instance profiles) are discarded during traversal.

To eliminate the opacity of black-box risk algorithms [24], [25], we compute asset risk via an inspectable linear heuristic:

$$R = \alpha S + \beta C + \gamma P + \delta E, \tag{1}$$

where all variables are normalized to $[0, 1]$:

- S: Raw finding severity assigned by evaluated benchmark rules.
- C: Criticality score reflecting data sensitivity and business role.
- P: Path metric reflecting the existence and length of verified candidate paths.
- E: Exposure metric capturing public routability and ingress accessibility.

The weighting parameters satisfy $\alpha + \beta + \gamma + \delta = 1$ and are frozen prior to evaluation. Exposure E is isolated from C to avoid double-counting external exposure in asset valuation. R serves as a comparative prioritization index rather than an empirical probability of breach.

TABLE II
CLAIM-TO-EVIDENCE MAP FOR EVALUATION

Claim	Required evaluation evidence and limitation
Finding detection	Scenario ground truth, normalised scan output, and finding-level precision/recall; limited to selected rules and scenarios.
Candidate-path quality	Expected-path catalogue, edge evidence, and independent path review; graph connectivity alone is insufficient.
Prioritisation	Predefined reference ranking and sensitivity analysis; R is a heuristic, not compromise probability.
LLM-assisted reporting	Human rubric for evidence consistency, accuracy, clarity, and actionability; output remains advisory.

D. LLM-Assisted Reporting Contract

The reporting interface accepts only validated configuration facts, graph paths, and rule metadata. The generative prompt explicitly instructs the model to:

1) Separate confirmed facts from hypothetical attacker actions.
2) List missing preconditions that prevent path completion.
3) Map every remediation action to a specific rule finding.
4) Insert an explicit disclaimer stating human review is required.

Because prompt constraints cannot guarantee complete avoidance of hallucination [28], [29], LLM outputs remain purely advisory and are barred from directly executing remediation commands. Table II details the claim-to-evidence validation map.

IV. Controlled Evaluation Design

We evaluate the framework in a dedicated, isolated AWS laboratory tenancy. The environment operates under least-privilege service roles, hard billing quotas, and comprehensive tagging, adhering to design-science validation rigor [9]. Synthetic data is used across all storage and database fixtures, and insecure test states are restricted from public Internet exposure.

A. Ground Truth and Units of Analysis

To eliminate ambiguity during scoring, we establish three distinct analysis units:

- **Finding:** A discrete rule violation linked to an individual cloud resource.
- **Candidate Path:** An ordered sequence of evidence-backed graph edges connecting an entry condition to a target asset.
- **Consolidated Alert:** A grouped cluster of co-located findings associated with a common workload or path context.

Prior to each assessment run, a formal scenario catalogue (Table III) records the intended configuration, expected findings, negative controls, and permitted remediation steps. This catalogue serves as the baseline ground truth, frozen and independently reviewed prior to execution [6], [12].

B. Metrics and Path Review

Detection performance is benchmarked strictly against the scenario ground truth using standard classification metrics:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP+FP}, \\ \text{Recall} &= \frac{TP}{TP+FN}, \\ F1 &= \frac{2TP}{2TP+FP+FN}, \end{aligned} \tag{2}$$

alongside the false positive rate $FPR = FP/(FP+TN)$ where negative control fixtures establish true negatives (TN).

Candidate paths are evaluated under three criteria:

1) **Exact Match:** The sequence of traversed nodes, edge permissions, entry conditions, and target assets matches the ground-truth specification.
2) **Partial Match:** The discovered chain represents a valid sub-path or superset, with divergences explicitly documented.
3) **Invalid Path:** The chain introduces unverified transitions, absent permissions, or missing preconditions.

Alert deduplication is measured as the ratio of consolidated alert clusters to raw baseline findings. These metrics are tracked independently and never combined into misleading composite scores.

C. Execution Protocol and Reproducibility

Every evaluation run archives pre-scan configuration snapshots, resource inventories, execution logs, raw JSON tool outputs, graph exports, and post-experiment teardown receipts. Comparative benchmarks against baseline tools enforce identical account scopes, IAM role permissions, and assessment windows. Non-warm-up runs are repeated at least three times to report latency distributions across scanning and graph-generation phases. An in-memory fixture runner validates catalogue integrity prior to AWS API execution; fixture results are treated as unit tests and excluded from cloud benchmark statistics.

D. LLM-Report Review Rubric

Each generated report undergoes blind evaluation by at least two independent security reviewers across four dimensions:

- **Evidence Consistency:** Absence of unsupported claims or unverified assumptions.
- **Technical Accuracy:** Correct identification of AWS service semantics and IAM policy behaviors.
- **Clarity:** Concise, structured presentation suitable for incident response triage.
- **Actionability:** Practical, safe remediation advice free of destructive commands.

Reviewers score each category on a predefined 1–5 scale, with inter-rater agreement measured via Cohen's kappa.

E. Research Questions

The evaluation addresses four targeted questions:

- **RQ1 (Detection):** Does the rule analyzer reliably identify labelled findings across S1–S6 with zero false positives on negative controls?
- **RQ2 (Path Validity):** Do generated candidate paths satisfy expected precondition constraints without asserting phantom traversals?
- **RQ3 (Prioritization):** Does the linear scoring formula R correlate with expert reference rankings under sensitivity analysis?
- **RQ4 (Report Fidelity):** Do LLM-assisted summaries adhere to the evidence boundary without introducing hallucinations or unsafe commands?

V. Implementation Status and Baseline Deployment Evidence

The baseline testbed remains deployed in an isolated AWS account in the `ap-south-2` region, fully managed via Terraform. For the multi-stage S5 scenario, a private EC2 instance, synthetic S3 object, metadata-only secret, subnet, security group, and instance profile were temporarily deployed, scanned, and subsequently destroyed via Terraform. Table IV outlines the verified baseline state.

Read-only API verification confirmed this baseline state, with subsequent Terraform execution plans showing zero pending modifications. Initial collection passes generated one medium-severity finding for an ingress rule permitting `tcp/8443` from `0.0.0.0/0`. Crucially, the engine rejected asserting an attack path: the necessary reachability preconditions—an attached Internet Gateway, an external route table entry, and an active network interface—were entirely absent.

TABLE III
CONTROLLED AWS SCENARIO CATALOGUE

ID	Scenario	Evaluation focus
S1	Public S3 storage exposure	Finding detection, evidence capture, and remediation guidance for a test-only bucket [13].
S2	Overly permissive IAM policy	Detection of test-principal permissions and supported identity relationships [8].
S3	Insecure network configuration	Detection of a laboratory security-group condition and documented reachability preconditions [18].
S4	Disabled CloudTrail logging	Detection of an audit-logging condition and restoration recommendation [4].
S5	Multi-stage candidate path	Evidence-backed relationship reasoning across test EC2, IAM, Secrets Manager, and S3 resources [19].
S6	SSRF-to-IAM credential-access candidate path	Planned local precondition-only analysis of configuration and identity conditions; no deployed service, public exposure, production credentials, or metadata requests.

TABLE IV
VERIFIED CONTROLLED-BASELINE STATE

Component	Observed configuration verification
S3 controls	All four S3 public-access blocks enabled; AES-256 default encryption enabled; the temporary S5 synthetic object was removed after measurement.
IAM role	Dedicated role has no attached managed policies; after S5 teardown its inline policy is limited to bucket listing/object read.
Network boundary	VPC has no Internet gateway or external route; the temporary S5 private subnet was removed after measurement.
S5 measurement	One private `t3.micro` instance was assessed without a public address; IMDSv2 required with hop limit one; encrypted root volume; all S5-only resources were then removed.
Security-group fixture	The S3 fixture has one `tcp/8443` ingress rule from `0.0.0.0/0`; the temporary S5 group had no ingress or egress.

In a secondary test pass reflecting S1/S2 conditions, bucket public-access blocks were temporarily lifted and an inline `s3:*` policy was attached to the test role. The evaluator detected both high-severity configuration anomalies but still correctly refused to generate an external attack path. In this instance, an account-level S3 Block Public Access setting intervened to block the public bucket policy, demonstrating defense-in-depth enforcement. For S4, a dedicated CloudTrail trail was deployed in a stopped state (`IsLogging = false`); the collector detected the missing audit trail without disrupting account-level logging. During the temporary S5 run, the engine observed the instance profile's read privilege toward the synthetic bucket while confirming the absence of a public IPv4 address, lack of Internet routing, IMDSv2 enforcement (hop limit 1), and denial of `secretsmanager:GetSecretValue`. Consequently, no compromise path was asserted. A deterministic local S6 model similarly rejected hypothetical SSRF paths in the absence of IMDSv1 or external egress. These results validate our precondition-filtering mechanics on live AWS infrastructure, while comparative benchmark trials remain planned future work.

VI. DISCUSSION AND LIMITATIONS

Our experimental findings reinforce that topological graph reachability must not be confused with true exploitability. Threat models, policy evaluation semantics, and traversal constraints must be explicitly formalized. Furthermore, our synthetic test scenarios do not encompass the full architectural diversity of enterprise multi-account environments [6], [7]. Expanding graph analytics beyond AWS to multi-cloud ecosystems [4], [5] requires unifying divergent provider permission semantics and entity models. In-memory graph engines also risk performance degradation in tenants containing millions of cloud assets, necessitating future research into distributed graph indexing [18].

Finally, deploying LLMs in cybersecurity workflows introduces latency, cost, and data governance considerations. Sensitive cloud resource identifiers and credential names must be sanitized prior to model ingestion, and generative output must never trigger autonomous remediation without human verification.

VII. CONCLUSION

This paper presented CloudSentinel AI, an AWS security framework designed to transition cloud misconfiguration analysis from isolated compliance audits to context-aware attack-path reasoning. By combining metadata collection, precondition-gated graph traversals, an explainable scoring heuristic, and evidence-locked LLM reporting, the framework targets the root causes of alert fatigue. Baseline tests on active AWS infrastructure confirm the framework's ability to capture verified configuration evidence and eliminate phantom attack paths. Future work will center on executing the full S1–S6 benchmark protocol, benchmarking detection against commercial CSPM baselines, and scaling the graph engine across multi-account cloud fabrics.

REFERENCES

[1] P. Mell and T. Grance, "The nist definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. SP 800-145, 2011.

[2] B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, and M. Ayaz, "A systematic literature review on cloud computing security: Threats and mitigation strategies," *IEEE Access*, vol. 9, pp. 57 792–57 807, 2021.

[3] R. Kumar and R. Goyal, "On cloud security requirements, threats, vulnerabilities and countermeasures: A survey," *Computer Science Review*, vol. 33, pp. 1–48, 2019.

[4] K. A. Torkura, M. I. Sukmana, F. Cheng, and C. Meinel, "Continuous auditing and threat detection in multi-cloud infrastructure," *Computers & Security*, vol. 102, p. 102124, 2021.

[5] D. Saxena, R. Gupta, and A. K. Singh, "A survey and comparative study on multi-cloud architectures: Emerging issues and challenges for cloud federation," *arXiv preprint arXiv:2108.12831*, 2021.

[6] Y. Liu, Y. Zhou, H. Zhang, Z. Chang, S. Xu, Y. Jia, W. Wang, and Z. Liu, "Rethinking software misconfigurations in the real world: An empirical study and literature analysis," 2024.

[7] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.

[8] T. van Ede, N. Khasuntsev, B. Steen, and A. Continella, "Detecting anomalous misconfigurations in aws identity and access management policies," in *Proceedings of the 2022 ACM Cloud Computing Security Workshop*, 2022, pp. 63–74.

[9] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[10] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: A survey," in *IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, 2022, pp. 218–225.

[11] D. B. Bose, A. Rahman, and M. S. I. Shamim, "'under-reported' security defects in kubernetes manifests," in *2021 IEEE/ACM 2nd International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, 2021, pp. 9–12.

[12] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.

[13] T. J. Olorunlana, "Preventing amazon s3 cloud storage misconfiguration using infrastructure-as-code: A policy-enforced security framework," *International Journal of Networked and Distributed Computing*, 2026.

[14] A. Hogan, E. Blomqvist, M. Cochez, C. D'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, 2021.

[15] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proceedings of the 1998 Workshop on New Security Paradigms*, 1998.

[16] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, 2002.

[17] S. Jha, O. Sheyner, and J. M. Wing, "Two formal analyses of attack graphs," in *Proceedings of the 15th IEEE Computer Security Foundations Workshop*, 2002.

[18] A. Sabur, A. Chowdhary, D. Huang, and A. Alshamrani, "Toward scalable graph-based security analysis for cloud networks," *Computer Networks*, vol. 206, p. 108795, 2022.

[19] M. Elmiger, M. Lemoudden, N. Pitropakis, and W. J. Buchanan, "Start thinking in graphs: Using graphs to address critical attack paths in a microsoft cloud tenant," *International Journal of Information Security*, vol. 23, no. 1, pp. 467–485, 2024.

[20] K. Kurniawan, A. Ekelhart, E. Kiesling, G. Quirchmayr, and A. M. Tjoa, "KRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data," *Computers & Security*, vol. 121, p. 102828, 2022.

[21] A. B. Nassif, M. Abu Talib, Q. Nasir, H. Albadani, and F. M. Dakalbab, "Machine learning for cloud security: A systematic review," *IEEE Access*, vol. 9, pp. 20 717–20 735, 2021.

[22] G. Rjoub, J. Bentahar, O. Abdel Wahab, R. Mizouni, A. Song, R. Cohen, H. Otrok, and A. Mourad, "A survey on explainable artificial intelligence for cybersecurity," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 5115–5140, 2023.

[23] F. Charmet, H. C. Tanuwidjaja, S. Ayoubi *et al.*, "Explainable artificial intelligence for cybersecurity: A literature survey," *Annals of Telecommunications*, vol. 77, pp. 789–812, 2022.

[24] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?: Explaining the predictions of any classifier," in *Proceedings of NAACL-HLT Demonstrations*, 2016, pp. 97–101.

[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[27] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[28] OpenAI, "Gpt-4 technical report," 2023.

[29] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," 2024.